Cloud Computing / Amazon Web Services

Amazon CloudWatch: EC2 Instance Monitoring, CloudWatch Alarms, SNS Notifications and Custom Dashboard Creation

Practical Laboratory Report

This practical explores EC2 performance metrics in Amazon CloudWatch, configures a CPU threshold alarm with email notification and a separate EC2 stop action, and creates and shares a dashboard for visual monitoring.

Practical topic: Amazon CloudWatch

Date: November 4, 2026

1 Objective

The objective is to use Amazon CloudWatch to monitor an existing Amazon EC2 instance and explore how a metric, an alarm, an Amazon Simple Notification Service (SNS) topic, and a dashboard work together. The practical selects the instance's `CPUUtilization` metric, configures a static threshold of 70% using the Average statistic over five-minute periods, and configures an email notification and an EC2 stop action for the alarm's `ALARM` state.

The practical also creates a custom dashboard with a CPU graph and explores public sharing. Monitoring collects and displays measurements, alerting evaluates measurements against a condition, SNS delivers messages to subscribers, and a dashboard presents selected measurements in one view. These are related but distinct functions.

2 Introduction to Amazon CloudWatch

Amazon CloudWatch is AWS's monitoring and observability service. It collects and presents operational data from AWS resources and applications, including metrics, logs, and alarms. In this practical, the focus is on EC2 metrics, a metric alarm, an SNS email destination, and a dashboard. EC2 publishes standard instance metrics to CloudWatch in the `AWS/EC2` namespace. [1, 3]

A **metric** is a time-ordered measurement identified by a name and, where applicable, dimensions that identify what is being measured. A namespace groups related metrics; `AWS/EC2` is the namespace used for standard EC2 metrics. A statistic such as Average summarizes samples over a time period. A **datapoint** is a measured or calculated value for a metric at a point or period in time. CloudWatch can graph datapoints and evaluate them continuously.

An **alarm** watches a metric and compares its evaluated value with a configured condition. Its state communicates whether the condition is not being breached, is being breached, or cannot currently be determined from available data. Alarm actions may send a notification through SNS or, for eligible per-instance EC2 metrics, invoke an EC2 action. The notification and the EC2 action are configured independently: SNS delivers a message; SNS does not stop the instance. [6]

A **dashboard** is a customizable view made from widgets such as metric graphs, numbers, or text. It is for visualization and does not itself collect the metric or execute the alarm action. In this lab, the graph is a separate view of the same CPU metric being monitored by the alarm. [11]

In short, **monitoring** measures and displays system behavior, **alerting** evaluates measurements and changes alarm state or invokes configured actions, and **visualization** arranges data to make trends easier to inspect.

3 Prerequisites

Before starting, prepare the following:

- An active AWS account and access to the AWS Management Console.

- An existing EC2 instance that is running and whose metrics may be viewed.
- Permissions to view CloudWatch metrics and create alarms and dashboards, create or use SNS topics and subscriptions, and configure EC2 alarm actions. The exact IAM permissions depend on account policies; an administrator may need to grant them.
- An email inbox that you can access to confirm the SNS email subscription.

An AWS **Region** is a geographic area in which many AWS resources are managed. A **service-linked role** is an IAM role defined for an AWS service to carry out supported operations on a user's behalf. CloudWatch uses such a role for EC2 alarm actions. The user creating or editing the alarm may need permission to create the role the first time it is required. [8]

4 Practical Environment and Initial Configuration

The environment details recorded for this experiment are:

- **Region:** Asia Pacific (Mumbai), `ap-south-1`.
- **Instance name:** `Cloud-Is-Watching`.
- **Instance type:** `t3.micro`.
- **Availability Zone:** `ap-south-1b`.
- **Initial instance state shown:** Running.

The source EC2 screenshot shows the instance ID and public DNS, both redacted in the included figure. The existing instance is the resource whose per-instance performance metric is selected. The initial Running state describes the first observation only; a separate later state crop is documented in the Results section without attributing its cause.

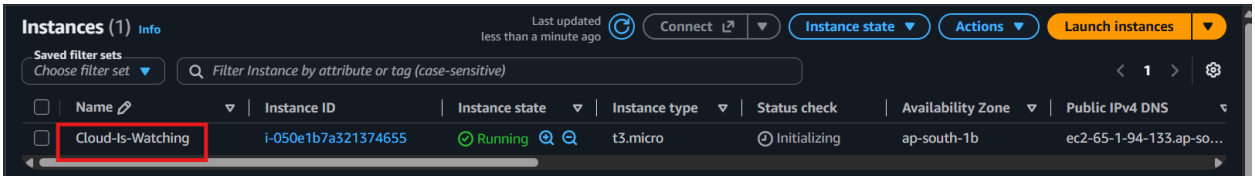


Figure 1: EC2 Instances page showing `Cloud-Is-Watching` initially Running in `ap-south-1b`; the instance ID and public DNS are redacted.

5 Implementation: Detailed Step-by-Step Procedure

5.1 Phase 1: Accessing Amazon CloudWatch

1. Sign in to the AWS Management Console using an authorized account.

2. Confirm that the console's Region selector shows **Asia Pacific (Mumbai)** (`ap-south-1`). If it does not, open the selector and choose this Region.

3. Select the AWS search box at the top of the console and enter `CloudWatch`.

4. In the service search results, choose **CloudWatch**. Wait for the CloudWatch console to load.

5. Review the CloudWatch navigation. For this experiment, the relevant areas are **Metrics** (to browse and graph measurements), **Alarms** (to create and review alarm states), and **Dashboards** (to create a visual overview). The console also provides other monitoring features, such as logs.

The screenshot sequence uses a CloudWatch service search result. Console labels and layout may change; in current CloudWatch navigation, metric browsing is generally under **Metrics** and **All metrics**. If a screenshot or account presents a **Classic metrics** label, follow that path for the shown interface; otherwise use the current metrics browsing page. CloudWatch is used here to discover the EC2 metric, graph it, evaluate its alarm condition, and create the dashboard. [1, 2]

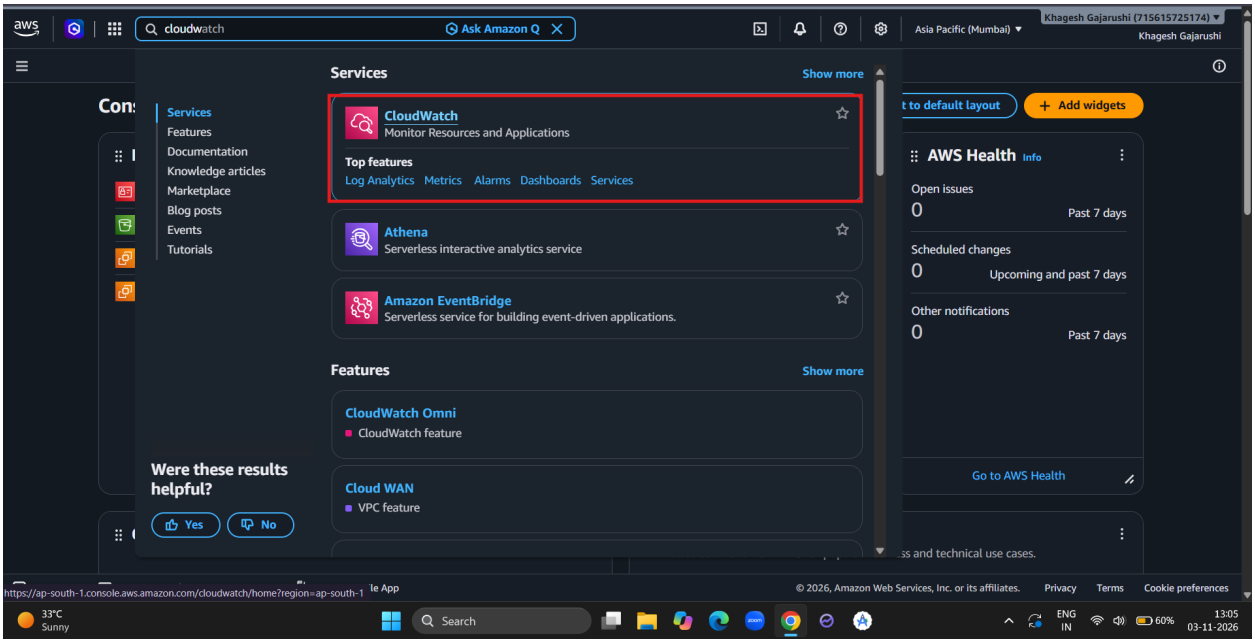


Figure 2: AWS service search results with CloudWatch selected. Account details are redacted.

5.2 Phase 2: Accessing EC2 Metrics

1. In CloudWatch, open the left navigation and choose **Metrics**. If the menu is collapsed, expand it first.

2. Choose **All metrics** or, in the screenshot's console layout, **Classic metrics**. Open the metric-browsing or **Browse** view.

3. Locate the metric search or filter field. Enter the EC2 instance ID shown on the EC2 Instances page. Filtering narrows a long list to metrics with that instance dimension. Do not copy the ID into a public report or screenshot.

4. Choose the **EC2** namespace, then choose **Per-Instance Metrics**. The exact order of filtering and choosing the dimension can differ slightly with the console version.

5. Review the metric names associated with the instance. The available metrics depend on the resource and its monitoring configuration.

Per-Instance Metrics are measurements identified for an individual EC2 instance, commonly by its `InstanceId` dimension. By contrast, **EBS Per-Volume Metrics** are measurements for individual Amazon Elastic Block Store volumes, such as volume read or write activity. Those volume measurements describe storage volumes rather than CPU performance of the instance. The EC2 namespace also has other dimensions and aggregate views; choosing the per-instance view helps avoid accidentally graphing an aggregate for multiple instances. [2, 3]

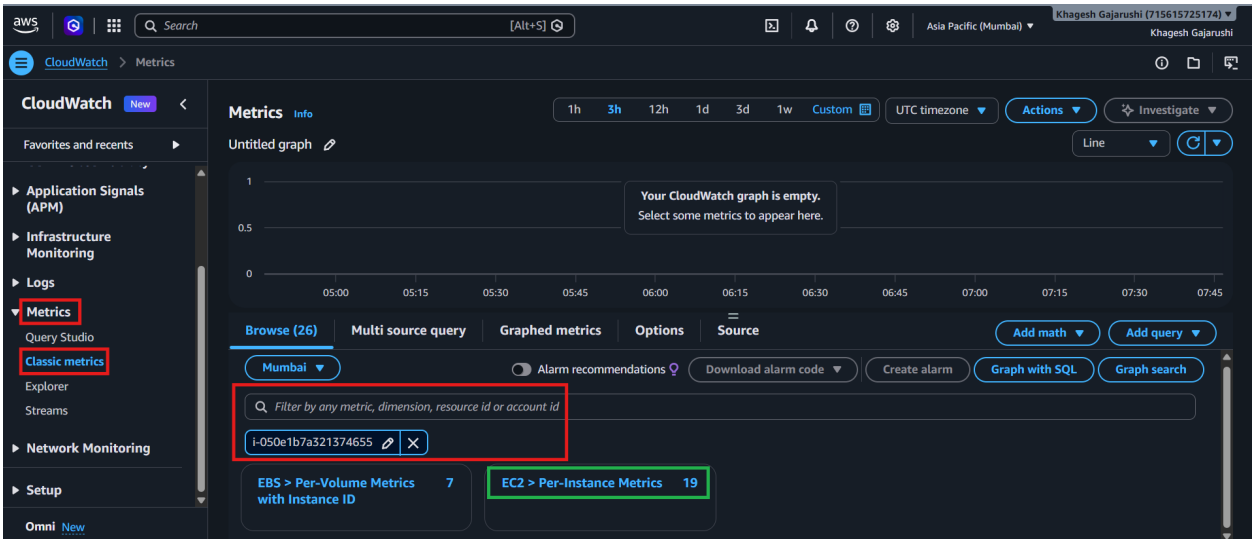


Figure 3: CloudWatch Classic metrics navigation, redacted instance-ID filter, and EC2 Per-Instance Metrics category.

5.3 Phase 3: Selecting `CPUUtilization`

1. In the selected instance's available metrics, locate `CPUUtilization`.

2. Select the checkbox beside `CPUUtilization`. CloudWatch adds the selected time series to the graph and to the **Graphed metrics** area.

3. Inspect the graph and its time-range control. A time range determines which period of metric history is displayed; the statistic and graph period determine how samples are summarized for display.

`CPUUtilization` is a percentage metric representing the physical CPU time used by the EC2 instance, including time spent running user code and EC2 code. It can help reveal sustained processor demand: a high value may indicate resource pressure or a workload that needs investigation. A

single high value is not, by itself, proof of a fault; interpret it in the context of the workload, duration, and application behavior. The metric is useful because it provides a standard instance-level measurement without requiring a separately installed guest-operating-system agent. [3]

EC2 basic monitoring generally publishes standard metrics at five-minute granularity; detailed monitoring provides one-minute granularity. The screenshots do not identify which monitoring mode was enabled for this instance, so its metric publication cadence cannot be determined from the images. The five-minute alarm period documented below is not proof of the instance's monitoring mode. [3]

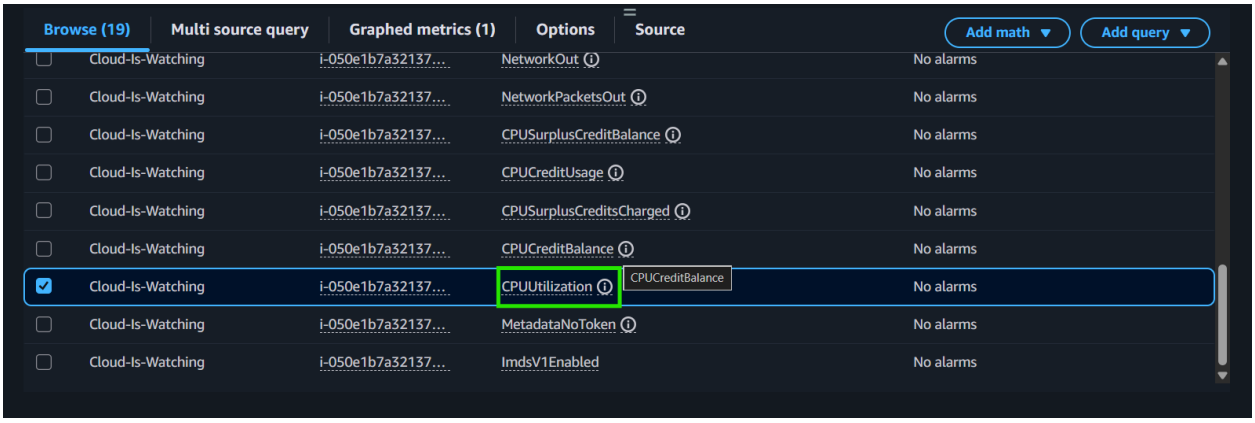


Figure 4: Available EC2 per-instance metrics with `CPUUtilization` selected; instance IDs are redacted.

5.4 Phase 4: Creating a CloudWatch Alarm

1. From the selected CPU metric or its graph, choose **Create alarm**. If the console instead requires an alarm-creation flow, choose **Alarms** and then **Create alarm**, and select the same metric.

2. Wait for the alarm configuration page to load. Review the metric summary before setting a condition.

3. Verify the namespace is `AWS/EC2` and the metric is `CPUUtilization`.

4. Verify the metric dimensions identify the intended instance. The instance name may be shown as a tag or label; the underlying instance ID is the identifying dimension.

5. Set or verify the statistic as **Average** and the period as **5 minutes**.

Average summarizes the samples received during a period by calculating their mean. It is a useful way to evaluate overall CPU use for that interval rather than reacting to a single sample. The **period** is the duration used to group samples into each datapoint. A five-minute period matches the requested alarm configuration and the standard basic-monitoring cadence for EC2; if detailed monitoring is enabled, one-minute data may also be available. Average and five minutes are the values documented for this lab, not a universal best setting for every workload. [3, 4]

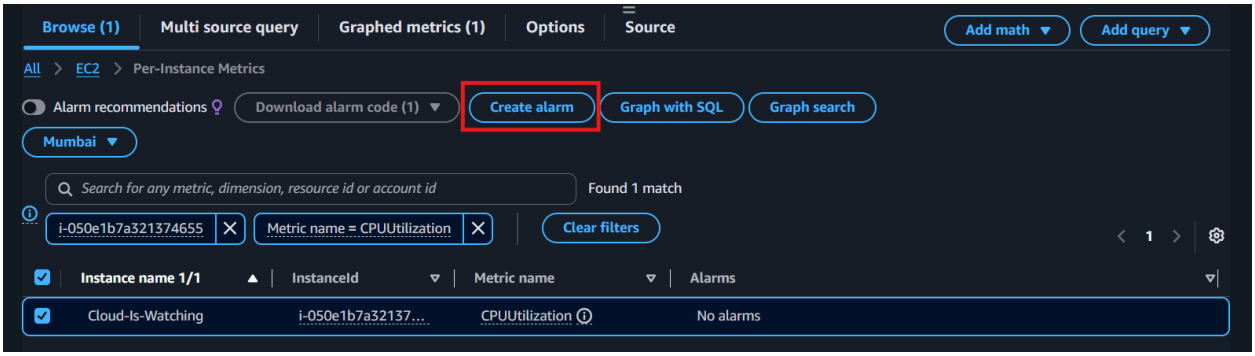


Figure 5: Selected EC2 `CPUUtilization` metric and the Create alarm control; the instance ID is redacted.

5.5 Phase 5: Configuring Alarm Conditions

Use the following values shown in the requested screenshots:

- **Threshold type:** Static.
- **Metric:** `CPUUtilization`.
- **Statistic:** Average.
- **Period:** 5 minutes.
- **Condition:** Greater than or equal to.
- **Threshold:** 70%.

A **static threshold** is a fixed value chosen by the user, as opposed to a threshold calculated from a learned baseline. For this alarm, the evaluated average is considered breaching when it is at least 70%.

The comparison choices mean:

- **Greater than:** breach only when the metric is above the threshold; equality does not breach.
- **Greater than or equal to:** breach when the metric is above or exactly equal to the threshold.
- **Less than or equal to:** breach when the metric is below or exactly equal to the threshold.
- **Less than:** breach only when the metric is below the threshold; equality does not breach.

The selected condition `CPUUtilization` $\geq$ `70%` marks high average CPU use at or above the chosen limit. Seventy percent is the experiment's threshold, not an AWS default or a universal definition of overload. According to the prompt, the alarm preview shows evaluation over **one datapoint within the five-minute period**. This is a one-of-one evaluation: one breaching datapoint is enough to make the condition true. The state still depends on the data CloudWatch receives and its missing-data handling. [4, 5]

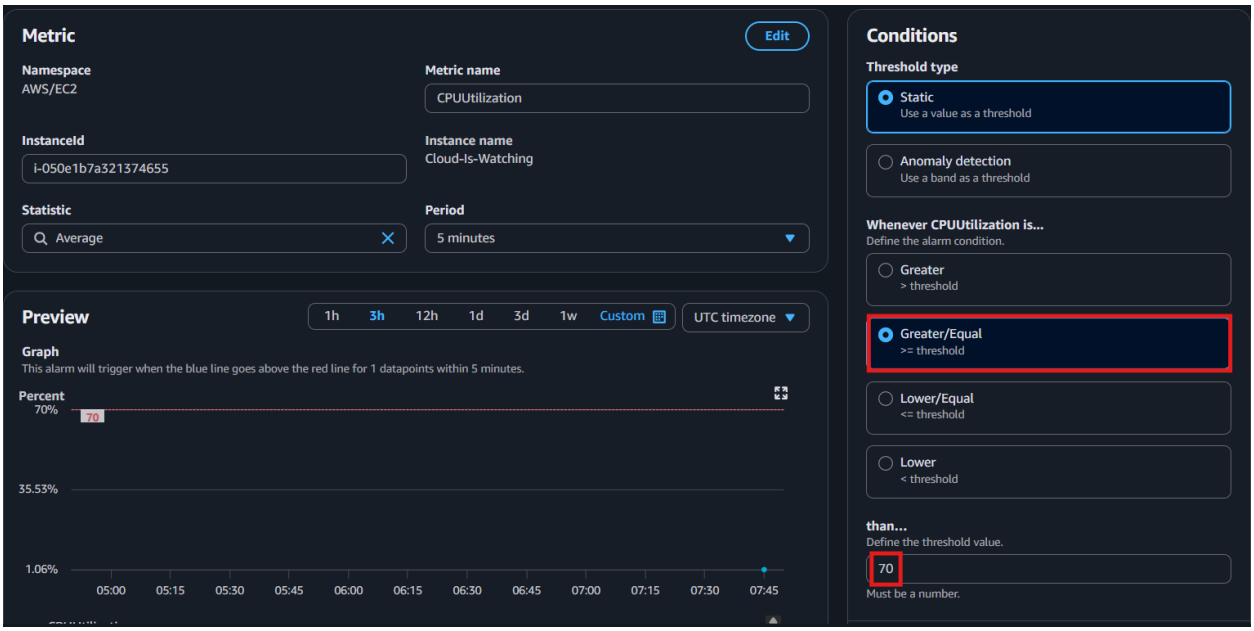


Figure 6: Alarm configuration showing `AWS/EC2`, `CPUUtilization`, Average, five minutes, Greater/Equal, and threshold 70%; the instance ID is redacted.

5.6 Phase 6: Configuring SNS Email Notifications

1. Continue to the alarm workflow's **Notification** section.
2. For the alarm-state trigger, select **In alarm**. This asks CloudWatch to send the configured notification when the alarm transitions into the `ALARM` state.
3. In the SNS topic selector, choose **Create new topic**.
4. Enter the topic name `Default_CloudWatch_Alarms_Topic`.
5. Enter the intended recipient's email address in the email endpoint field. Use an inbox you can access. The address is not included in this report.
6. Choose **Create topic** and continue with the alarm setup.

An SNS **topic** is a named distribution point that can deliver a message to one or more subscribed endpoints. CloudWatch publishes the alarm notification to the selected topic; SNS then sends it to the email subscription. Creating a topic and subscribing the email are logically separate steps, even when the CloudWatch wizard offers them together. The email subscription remains pending until the recipient confirms it. [10, 7]

5.7 Phase 7: Confirming the SNS Subscription

1. Open the inbox for the email address entered in the SNS endpoint field.
2. Find the Amazon SNS subscription confirmation email and open it. Check spam or filtered folders if it is not in the inbox.

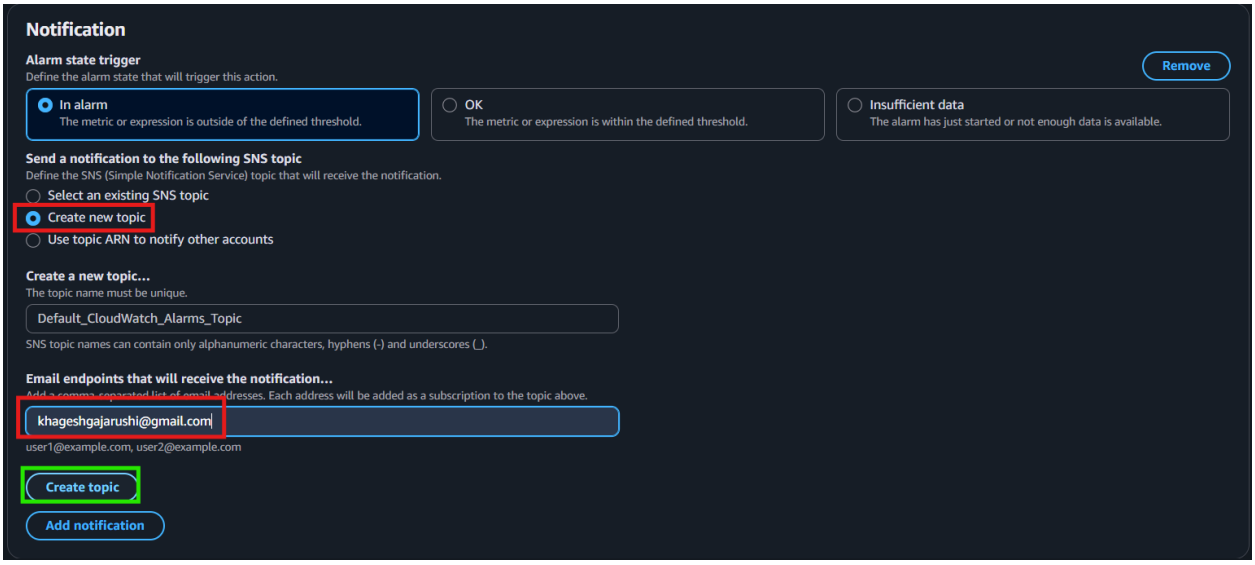


Figure 7: Notification settings with In alarm, Create new topic, `Default_CloudWatch_Alarms_Topic`, and Create topic; the email endpoint is redacted.

3. Choose **Confirm subscription** in the message.
4. Wait for the browser confirmation page and verify that it displays **Subscription confirmed!**.

Confirmation demonstrates that the endpoint owner controls the address and wants to receive messages from that topic. Until confirmation, the email subscription is pending and cannot receive topic notifications. The email screenshot should be cropped or redacted so the address, message identifiers, and any subscription token or URL are not disclosed. [10]

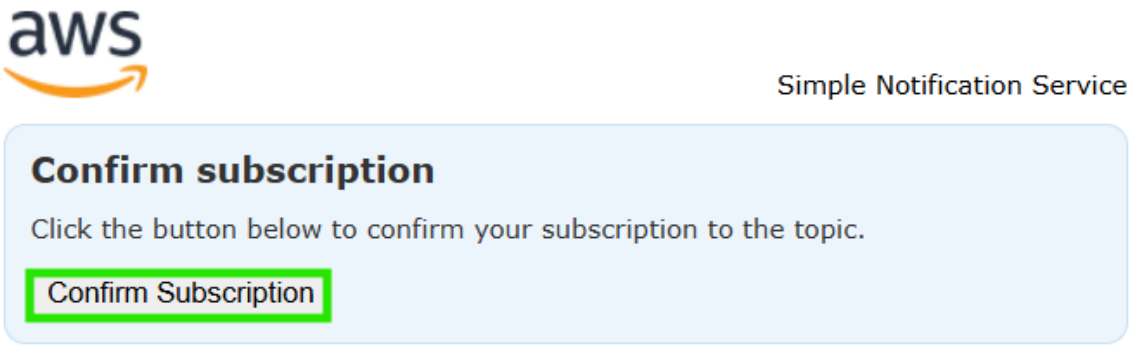


Figure 8: Amazon SNS email asking the endpoint owner to confirm the subscription.

5.8 Phase 8: Configuring Automated EC2 Actions

1. Return to the CloudWatch alarm-creation workflow and locate the **EC2 action** section.

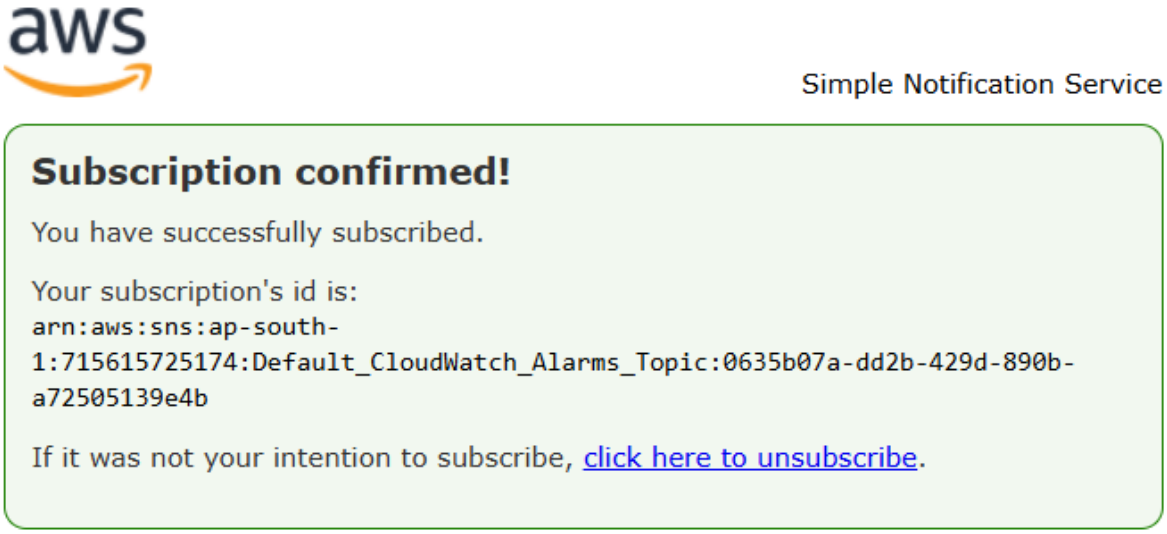


Figure 9: SNS page showing Subscription confirmed; the subscription ARN and account identifier are redacted.

2. Set its alarm-state trigger to **In alarm**.
3. Review the available actions and choose **Stop this instance**.
4. Choose **Add EC2 action** to attach the action to the alarm configuration.

The selected option configures CloudWatch to request that EC2 stop the instance if the alarm enters `ALARM`. The action-configuration screenshot does not show an alarm transition or prove that the configured action executed. A separate screenshot crop shows the instance as Stopped, but it does not identify the cause; do not attribute that state to this alarm.

The screenshot shows **Recover this instance** as unavailable for this configuration. Availability depends on instance eligibility and the alarm context. The other actions have different effects:

- **Recover:** attempts to recover an eligible instance after a supported system impairment. Recovery is not available for every instance or metric; it is commonly associated with a system status-check alarm.
- **Stop:** stops the instance. A stopped instance can generally be started again, retaining its instance identity and EBS-backed root volume; data on instance-store volumes is lost.
- **Terminate:** permanently terminates the instance; it cannot be started again. Data on instance-store volumes is lost, and the user must launch a new instance if one is needed.
- **Reboot:** restarts the instance operating system, similar to a normal reboot, and is distinct from stopping or replacing the instance. In-memory state is not preserved through a reboot.

This lab explicitly selects **Stop**, not **Terminate**. CloudWatch uses its service-linked role, named `AWSServiceRoleForCloudWatchEvents`, for supported EC2 alarm actions; suitable IAM permissions are required to create or use the role. [9, 8]

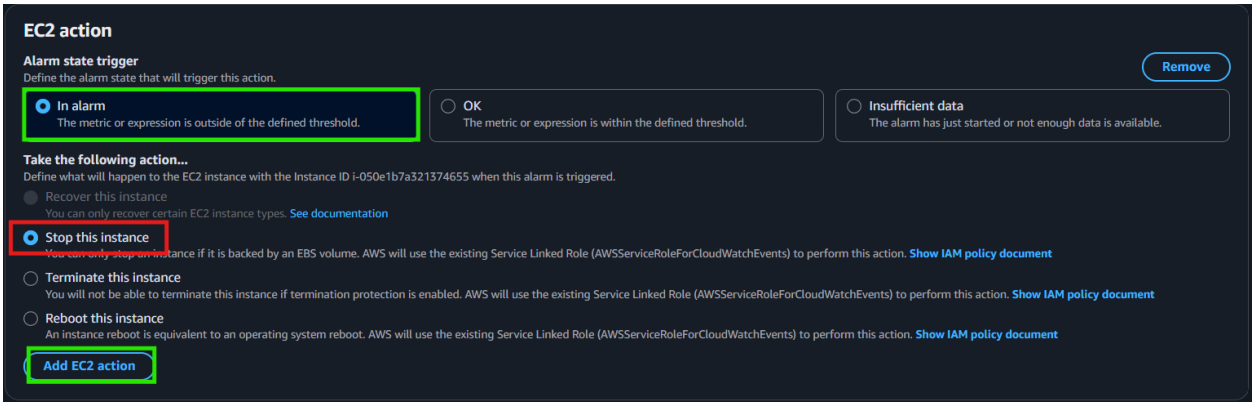


Figure 10: EC2 action settings showing In alarm and Stop this instance; the instance ID is redacted.

5.9 Phase 9: Creating and Reviewing the Alarm

Review the alarm configuration summary and choose **Create alarm** (or the equivalent final create control displayed by the console). The alarm appears in the alarm list under the name `OHH-MY-GAWDD`. The alarm-list screenshot shows its state as **Insufficient data**, with **Actions enabled** and a visible **Warning** indicator.

CloudWatch metric alarms use three main states:

- **OK:** the available evaluated datapoints are not breaching the configured condition.
- **ALARM:** the configured number of evaluated datapoints breach the condition.
- **INSUFFICIENT_DATA:** CloudWatch does not have enough usable data to determine whether the alarm is OK or ALARM.

A new alarm may initially show `INSUFFICIENT_DATA` while the evaluation window is being populated, when the metric is missing or delayed, or when there are too few usable datapoints. That state is not a failure by itself and is not proof that CPU reached 70%. For EC2 alarms with actions, treating missing metric data as a breach can produce unintended behavior; the missing-data setting should be reviewed carefully. `Actions enabled` indicates that alarm actions are enabled in the configuration; it does not show that an action has executed. [5, 4]

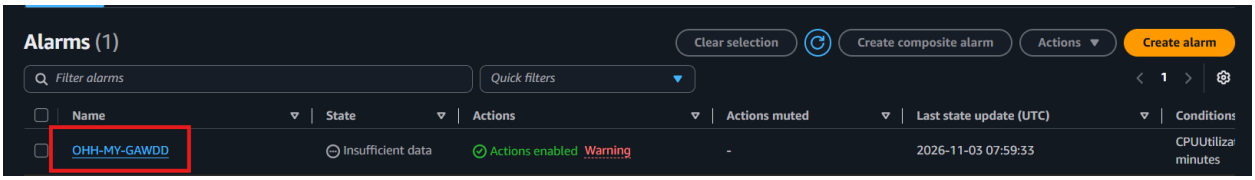


Figure 11: Alarm list showing `OHH-MY-GAWDD`, Insufficient data, Actions enabled, and the Warning indicator.

5.10 Phase 10: Creating a Custom CloudWatch Dashboard

1. In the CloudWatch console navigation, open **Dashboards**.
2. Review the **Custom Dashboards** list. The requested initial view shows that there were no custom dashboards before this practical.
3. Choose **Create dashboard**.
4. Enter `HOOLA-HOOO` as the dashboard name and continue through the creation dialog.
5. When prompted to add content, choose a metric graph widget and configure it in the following phase. Save the dashboard when the widget is ready.

A custom dashboard collects selected views into a central monitoring screen. Each **widget** is one panel, such as a metric graph. Dashboards are global resources rather than being limited to a single Region, although a widget's metric selection still refers to its relevant Region and metric dimensions. [11]

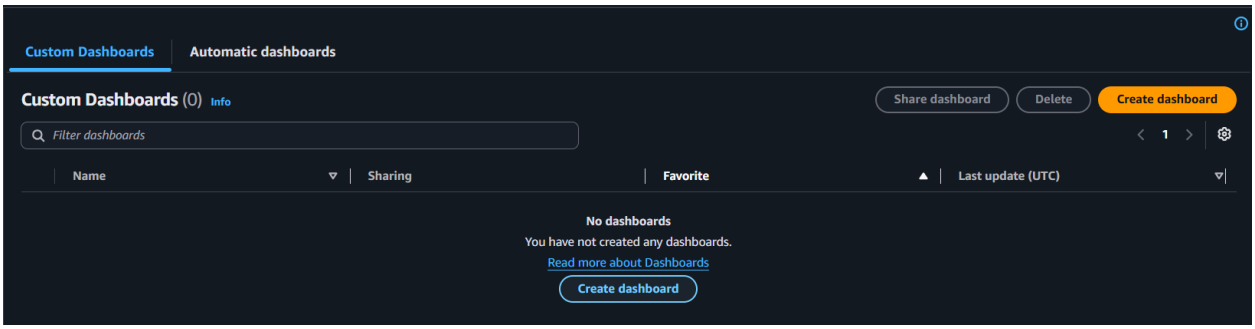


Figure 12: Custom Dashboards list before creation, showing no custom dashboards.

5.11 Phase 11: Adding `CPUUtilization` to the Dashboard

The experiment details specify the dashboard name `HOOLA-HOOO`; however, the supplied dashboard screenshots appear to show `HOOLA-HOO` (one fewer trailing O). This naming discrepancy is retained rather than silently corrected; verify the intended name before submitting the final report. The captured dashboard contains a CPU metric graph. To add or verify the metric in the creation flow:

1. In the dashboard's add-widget dialog, select a line or area metric graph and choose its configuration control.
2. Browse to `AWS/EC2` and **Per-Instance Metrics**, then select `CPUUtilization` for the lab instance.
3. Create the widget and save the dashboard. Verify the displayed metric is `CPUUtilization` and its unit is **Percent**.
4. Use the dashboard's time-range control to inspect another supported time window. Changing the time range changes the period of history shown, not the underlying metric definition.

The metric widget turns the selected measurements into a visual time series. A graph makes changes and trends easier to see than a list of values, but it does not by itself establish a threshold breach or execute an alarm action. The captured widget identifies `CPUUtilization` in Percent; the screenshots show a varying graph but do not establish that the alarm threshold was breached. [11, 3]



Figure 13: Dashboard CPUUtilization graph with unit Percent. The captured dashboard heading appears as `HOOLA-HOO`.

5.12 Phase 12: Sharing the CloudWatch Dashboard

1. Open the created dashboard from the CloudWatch **Dashboards** list. Its heading appears as `HOOLA-HOO` in the screenshots, while the experiment details specify `HOOLA-HOOO`; verify the intended spelling.
2. Open the dashboard's **Actions** menu and choose **Share dashboard**.
3. Find the public-sharing section labeled **Share your dashboard publicly** and choose **Start sharing**.
4. In the confirmation dialog, type `Confirm`, read the agreement, select the confirmation checkbox, and choose **Preview policy**.
5. Review the resources listed in the policy. If the listed access is appropriate, choose **Confirm and generate shareable link**.
6. On the next page, verify the success message and choose **Copy link** only if the dashboard is intentionally being distributed.

Public sharing means anyone with the dashboard link can access it without AWS sign-in. This is a significant security exposure: AWS documents that public viewers receive permissions that can include querying CloudWatch metrics and viewing EC2 names and tags, not only seeing the particular graph. Do not share account identifiers, personal email addresses, subscription ARNs, sensitive metric names, operational details, or other private information. Review the resources and sharing policy before confirming. Enable public sharing only when the dashboard is appropriate for public access, keep the link controlled, and choose **Stop sharing** when it is no longer needed. The screenshots show a success notification and Copy link option; the actual public link is not reproduced. [12, 13]

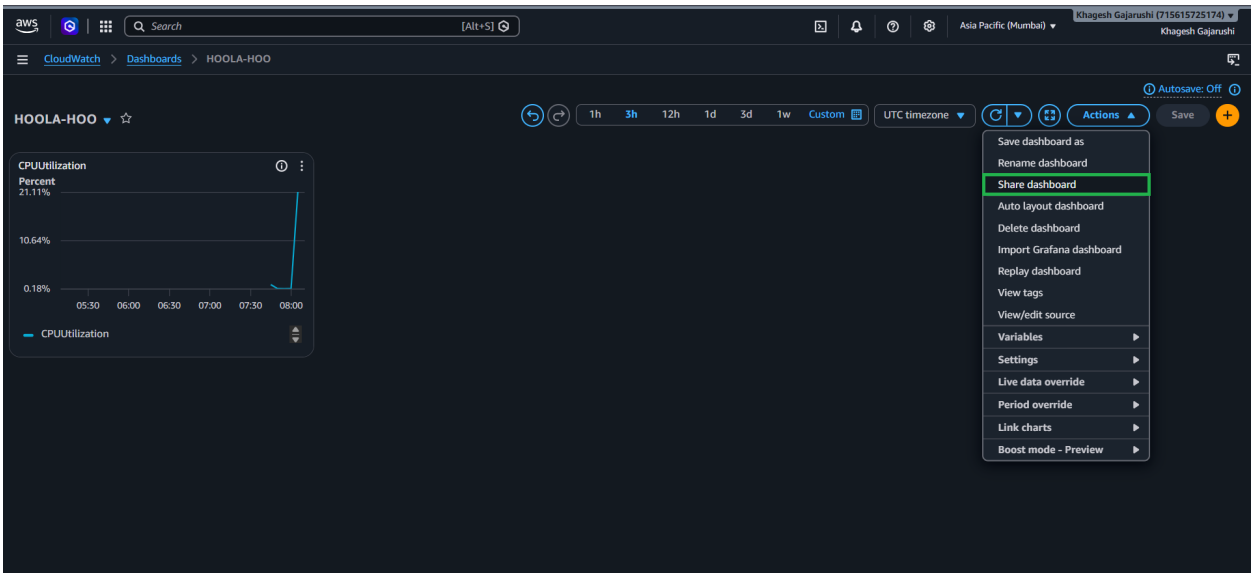


Figure 14: Dashboard Actions menu with Share dashboard selected; account details are redacted.



Figure 15: Public-sharing section showing Share your dashboard publicly and Start sharing.



Figure 16: Success notification showing public sharing was initiated and Copy link is available. The dashboard appears as `HOOLA-HOO`.

6 Overall Working Architecture

The monitoring and response paths are separate branches from the CloudWatch alarm. The metric branch feeds both the alarm and the dashboard. The notification branch uses SNS to deliver email, while the stop branch is an EC2 alarm action executed through CloudWatch's service-linked role.

The metric path is **EC2 instance → CloudWatch metrics → CPUUtilization → CloudWatch alarm**. The notification path is **CloudWatch alarm → SNS topic → confirmed email**

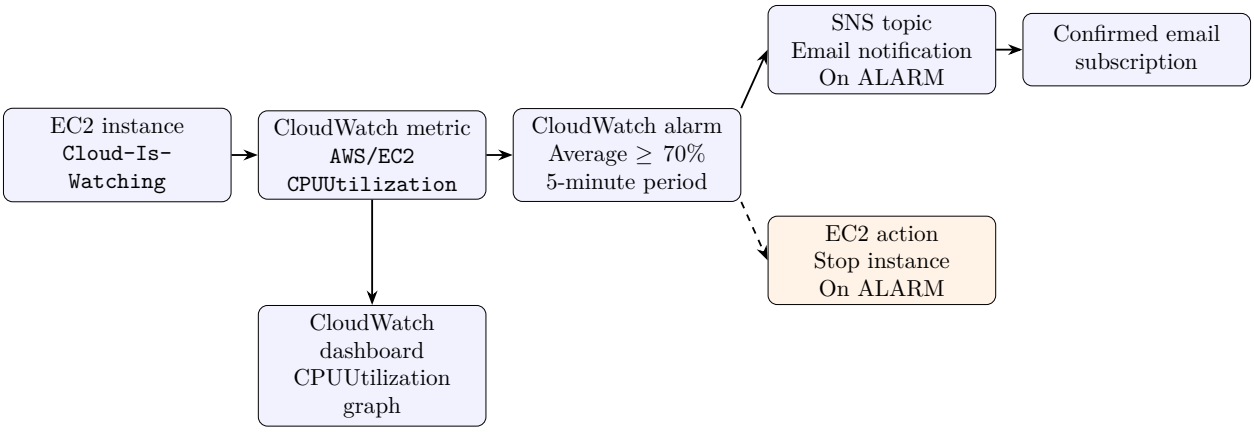


Figure 17: Monitoring, notification, visualization, and configured EC2-action paths. The dashed arrow marks an action configured for the alarm's ALARM state, not an action shown to have executed.

subscription. Separately, the configured action path is **CloudWatch alarm → EC2 stop action**. SNS is not part of the stop path. The dashboard is a visualization component that displays the collected metric.

7 Observations and Results

The observations below are supported by the inserted screenshots and the experiment details. Personal or account identifiers have been redacted in the included screenshot copies.

1. The EC2 instance was available in the Mumbai Region (`ap-south-1`); its initial screenshot state was Running.
2. CloudWatch displayed EC2 per-instance metrics for the selected instance.
3. `CPUUtilization` was selected for monitoring.
4. An alarm was configured with a 70% static threshold, Average statistic, five-minute period, Greater/Equal comparison, and one datapoint as indicated in the alarm preview.
5. The SNS topic `Default_CloudWatch_Alarms_Topic` was created for an email notification.
6. The email subscription was confirmed; the reported confirmation page displayed **Subscription confirmed!**.
7. A stop action was configured for the alarm's `ALARM` state. The available evidence does not show that this action executed.
8. The alarm `OHH-MY-GAWDD` appeared in the alarm list with state `INSUFFICIENT_DATA` and actions enabled.
9. A custom dashboard with a CPUUtilization graph in Percent was created. The supplied details specify `HOOLA-HOOO`, while its screenshots appear to show `HOOLA-HOO`; verify the exact name.

10. Public dashboard sharing was initiated successfully, as shown by the success notification and Copy link option. The actual public link is intentionally not reproduced.

These are configuration and interface observations. They do not establish that CPU reached or exceeded 70%, that the alarm entered `ALARM`, that an alarm email was sent because of a breach, or that the configured alarm action caused an EC2 stop. A separate EC2 state crop (Figure 18) shows `Stopped`, but it does not show why the instance stopped. A confirmed subscription proves the endpoint confirmation step, not that a breach notification was delivered.



Figure 18: Separate EC2 console crop showing the instance state as Stopped. It does not identify the cause or demonstrate that the CloudWatch alarm performed the stop.

8 Additional Guided Tasks: Integrated CloudWatch Automation Architecture

Implementation status:
NOT IMPLEMENTED / THEORETICAL WALKTHROUGH.

The original hands-on practical remains unchanged, including its configured 70% alarm, SNS notification, and **Stop this instance** action. This addendum describes one separate, integrated architecture with two independent CPU alarms: a high-CPU alarm that scales out an Auto Scaling group and sends email, and a low-CPU alarm that sends email only. Nothing in this section claims that an Auto Scaling group or either of these two alarm configurations was created or tested.

8.1 Integrated architecture and scope

Both alarms in this proposed design monitor the same Auto Scaling group workload using its EC2 `CPUUtilization` metric. The high alarm has two independent responses: it invokes a step scaling policy and sends an SNS notification. The low alarm has only an SNS notification; it has no scale-in, stop, reboot, or termination action.

Important scope distinction. The original instance `Cloud-Is-Watching` is a standalone EC2 instance in the practical. It does not become a member of an Auto Scaling group merely because an alarm has an Auto Scaling action. To use this integrated design, deploy the application workload in an Auto Scaling group and select that group's CPU metric. Do not simply reuse the practical's alarm with its EC2 stop action: that is a different response and could conflict with scaling. The

existing practical result and configured stop action remain documented as they were; this theoretical architecture does not modify them.

A launch template defines how the Auto Scaling group launches EC2 instances. The Auto Scaling group manages desired capacity within its minimum and maximum limits. A CloudWatch alarm evaluates CPU datapoints for that group and invokes the configured action or actions when its condition is met. Amazon SNS delivers the notification to confirmed email subscribers; SNS does not perform scaling.

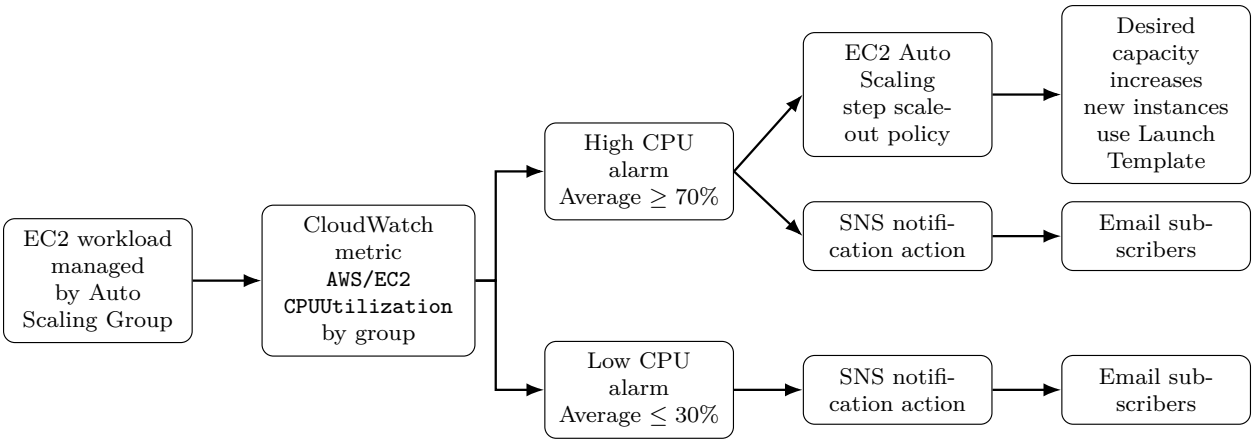


Figure 19: Integrated guided architecture. The 70% alarm has both a scale-out response and an SNS notification; the 30% alarm sends an SNS email only. The SNS actions may use one shared topic or separate topics.

8.2 Scenario A: High CPU alarm at 70%

When the group's evaluated Average `CPUUtilization` reaches or exceeds 70%, the high alarm can enter `ALARM` after its configured evaluation criteria are met. Its step scaling action increases the Auto Scaling group's desired capacity, allowing EC2 Auto Scaling to launch additional instances from the launch template, subject to the group's maximum capacity. The same alarm sends an SNS email notification. These are separate alarm actions on the same high-CPU alarm.

8.2.1 Prepare the launch template and Auto Scaling group

1. Sign in to the AWS Management Console and select the Region in which this guided workload will run. Keep the launch template, Auto Scaling group, CloudWatch alarms, and SNS topic in the intended Region.

2. Use the console search to open **EC2**. In the EC2 navigation pane, under **Instances**, choose **Launch Templates**, then choose **Create launch template**.

3. Enter a descriptive template name. In the launch template contents, choose an AMI containing the intended application, a compatible instance type, and the required key-pair, security group, storage, tags, IAM instance profile, and startup configuration. Review the settings carefully because the group uses this template when it launches new instances.

4. Create the launch template. From its confirmation page, choose **Create Auto Scaling group**, or open **Auto Scaling Groups** in the EC2 navigation pane and choose **Create Auto Scaling group**.

5. Select the launch template and version. Choose the VPC that matches the template's security groups, then choose suitable subnets. For an application that needs availability across zones, select appropriate subnets in multiple Availability Zones. The Auto Scaling group, rather than the launch template, specifies the subnets used for the group.

6. If the application receives network traffic, configure an appropriate load balancer or target group so that traffic can be distributed to healthy instances. This is workload-dependent; a load balancer is not required for every Auto Scaling group.

7. Set the group's minimum, desired, and maximum capacity. Minimum is the lower boundary, desired is the target number of instances to maintain initially, and maximum is the upper boundary. Choose values based on tested application capacity, availability, and budget; do not leave the maximum so low that the scale-out action cannot add capacity.

8. Review and create the Auto Scaling group. Confirm that its launch template, VPC, subnets, and capacity values match the design before continuing.

8.2.2 Create the high-CPU metric alarm and SNS notification

1. In the console search, open **CloudWatch**. In its navigation pane, choose **Alarms**, then **All alarms**, and choose **Create alarm**.

2. Choose **Select metric**. On the metric browser, choose **EC2**, then **By Auto Scaling Group**. Find the intended group and select its `CPUUtilization` metric. Verify that the `AutoScalingGroupName` dimension identifies the intended group. This group-level metric ties both guided alarms to the same workload instead of to an unrelated standalone instance.

3. Set the statistic to **Average** and the period to **5 minutes**. These are the illustrative settings used in the original practical; select different values only if the workload and monitoring requirements call for them.

4. Under conditions, select **Static**. Set the comparison to **Greater than or equal to** (the console may show **Greater/Equal**) and enter `70` as the threshold.

5. Review the evaluation period and datapoints-to-alarm settings. These determine how many recent periods must breach before the alarm enters `ALARM`; choose them to avoid reacting to a brief spike if sustained high CPU is the intended signal.

6. Choose **Next**. In the notification settings, add an SNS notification for the **In alarm** state. Select an existing appropriate SNS topic or create a topic and enter the intended email endpoint. If creating a new email subscription, the recipient must confirm it before messages can be delivered.

7. Create the CloudWatch alarm with the SNS notification configured. At this point the alarm can notify subscribers, but the Auto Scaling policy still needs to be associated in the next procedure.

8. Open the registered email inbox, locate the SNS subscription confirmation request, and choose **Confirm subscription** if the endpoint is new or pending. Verify the confirmation page before relying on email delivery.

8.2.3 Create and associate the scale-out policy

1. Return to the EC2 console and open **Auto Scaling Groups**. Select the group created for this workload.
2. Open the group's **Automatic scaling** tab. Under **Dynamic scaling policies**, choose **Create dynamic scaling policy**.
3. Choose **Step scaling** and enter a descriptive policy name. In the CloudWatch alarm field, select the high-CPU alarm created above.
4. Under the step adjustment, choose a positive capacity change, such as adding a chosen number of instances or a chosen percentage of the group. The adjustment amount is workload-specific; select and test it so that it is useful but bounded by the group's maximum capacity.
5. Configure the instance warmup period for the time the new instances need before they contribute useful capacity and metrics. Review the alarm evaluation and group scaling limits as well.
6. Create the policy. The Auto Scaling console associates the step scaling policy with the selected CloudWatch alarm. Return to CloudWatch and verify that the high alarm retains its SNS notification and now also has the Auto Scaling scale-out action. Do not select the original practical's **Stop this instance** action for this design.
7. In a non-production environment, generate a controlled workload and review the CloudWatch alarm history, Auto Scaling activity history, group desired capacity, and instance health. This is a proposed validation procedure; no scaling test was performed for this document.

A step scaling policy defines how group capacity changes when an associated CloudWatch alarm breaches. The alarm evaluates the metric; the scaling policy applies the configured capacity adjustment. An alarm may have multiple actions, so the scale-out action and SNS notification can be configured for the high alarm together. Auto Scaling actions are an exception to the usual transition-only alarm-action behavior, while SNS notifications are associated with alarm state transitions. Policy behavior and group state determine whether additional capacity changes occur while the alarm remains in ALARM. Configure evaluation settings, step adjustments, warmup, and maximum capacity carefully to avoid unnecessary repeated scaling.

Target tracking is a different option. A target tracking policy asks Auto Scaling to maintain a chosen metric target and creates and manages its own CloudWatch alarms. Do not confuse those managed alarms with this guided design's explicit 70% CloudWatch alarm associated with a step scaling policy.

8.3 Scenario B: Low CPU alarm at 30% with email only

Create a separate alarm for the same Auto Scaling group's `CPUUtilization` metric. Its static condition is Average CPU utilization less than or equal to 30%. When its evaluation criteria are met, it sends an SNS email notification only. It has no Auto Scaling action and must not scale in, stop, reboot, or terminate instances.

1. In CloudWatch, choose **Alarms**, then **All alarms**, and choose **Create alarm**.
2. Choose **Select metric**, then browse to **EC2** and **By Auto Scaling Group**. Select the same group's `CPUUtilization` metric used by the high alarm.
3. Verify the group dimension, set the statistic to **Average**, and set the illustrative period to **5 minutes**.
4. Under conditions, choose **Static**, select **Less than or equal to** (or the console's **Less than/Equal** label), and enter `30` as the threshold.
5. Review evaluation periods and datapoints-to-alarm. These determine how many low datapoints are needed before the alarm changes state; configure them to match the monitoring goal.
6. Choose **Next**. In the notification section, set the trigger to **In alarm** and select an appropriate SNS topic, or create one with the intended email endpoint.
7. Do not add an EC2 Auto Scaling action or any EC2 stop, reboot, or termination action to this alarm. Its only configured response in this design is the SNS email notification.
8. Review and create the alarm. Confirm any new SNS email subscription using the confirmation message sent to that address.
9. Return to the CloudWatch alarm list and verify that the low alarm exists and that its actions contain SNS notification only.

Threshold versus crossing. A condition of `CPUUtilization` $\leq 30\%$ evaluates whether metric datapoints are at or below 30%; it does not mean the alarm fires only at the precise moment CPU crosses downward through 30%. If the alarm has sufficient breaching datapoints when first evaluated, it can enter `ALARM` without an observed downward crossing. Evaluation periods and datapoints-to-alarm determine the state transition. An SNS action configured for **In alarm** sends a notification on the alarm's transition into that state; inspect alarm history and metric datapoints to understand when that transition occurred.

8.4 Scaling behavior and important considerations

- **Scale-out** increases the number of instances in the Auto Scaling group. A scaling policy changes desired capacity, and the group uses its launch template to launch the additional instances, subject to the maximum capacity.
- **Scale-in** reduces group capacity. The 30% alarm in this design has no scale-in policy or action; low CPU prompts investigation only.

- **Capacity boundaries** matter. Desired capacity is the target group size, while minimum and maximum capacity constrain scaling. A maximum equal to the current desired capacity prevents scale-out.
- **Warmup and cooldown** settings help account for the time new instances need to start serving the workload and help avoid responding too aggressively to short-lived metric changes. The exact setting depends on the scaling policy and application startup time.
- **Group-managed instances** are launched, monitored, and replaced as members of the Auto Scaling group. The original standalone lab instance is not automatically managed by the new group.
- **Application design** must support multiple instances where needed. Keep durable state outside replaceable compute instances, and use a load balancer when incoming requests need distribution across healthy instances.
- **Cost and data safety** require limits and planning. Scale-out can increase EC2 charges. Scale-in terminates instances; data on instance-store volumes is not persistent across stop or termination, and EBS retention depends on volume settings. Back up important data and use appropriate persistent storage.

8.5 Final behavior comparison

Condition	**CloudWatch alarm**	**Auto Scaling**	**SNS notification**
CPU $\geq 70\%$	High CPU alarm	Scale-out policy increases desired capacity	Yes
CPU $\leq 30\%$	Low CPU alarm	No action	Yes
CPU between thresholds	Neither threshold condition is met after evaluation	No action from these alarms	No new threshold notification

The table describes evaluated steady-state conditions. CloudWatch uses datapoints, evaluation periods, and alarm state transitions, so the state may not change at the exact instant the underlying CPU value crosses a threshold. Auto Scaling actions are an exception to the usual transition-only behavior; scaling policy behavior and group state determine whether additional capacity changes occur while the high alarm remains in ALARM. SNS notifications are associated with state transitions. If the metric moves between thresholds, allow the configured evaluation window to determine when each alarm returns to `OK`.

8.6 Official AWS documentation for this guided design

- Amazon CloudWatch, *Create an alarm that triggers an Auto Scaling action*: `https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/US_AlarmAutoScalingAction.html`.

- Amazon EC2 Auto Scaling, *Create a step scaling policy for scale out*: https://docs.aws.amazon.com/autoscaling/ec2/userguide/step-scaling-create-scale-out-policy.html.
- Amazon EC2 Auto Scaling, *Create a launch template for an Auto Scaling group*: https://docs.aws.amazon.com/autoscaling/ec2/userguide/create-launch-template.html.
- Amazon EC2 Auto Scaling, *Create an Auto Scaling group using a launch template*: https://docs.aws.amazon.com/autoscaling/ec2/userguide/create-asg-launch-template.html.
- Amazon EC2 Auto Scaling, *Set scaling limits for your Auto Scaling group*: https://docs.aws.amazon.com/autoscaling/ec2/userguide/asg-capacity-limits.html.
- Amazon EC2 Auto Scaling, *Target tracking scaling policies*: https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-target-tracking.html.
- Amazon EC2 Auto Scaling, *Set the default instance warmup for an Auto Scaling group*: https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-default-instance-warmup.html.
- Amazon EC2 Auto Scaling, *Attach a load balancer to an Auto Scaling group*: https://docs.aws.amazon.com/autoscaling/ec2/userguide/attach-load-balancer-asg.html.
- Amazon EC2, *Data persistence for instance store volumes*: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instance-store-lifetime.html.

How the components work together. CloudWatch metrics monitor the EC2 workload, the two alarms evaluate distinct high- and low-CPU conditions, and their configured actions respond independently. For high CPU, the step scaling policy raises the Auto Scaling group's desired capacity while SNS informs email subscribers; for low CPU, SNS sends an alert without changing capacity. Together, monitoring, alerting, and automated capacity management provide visibility and a controlled response while leaving workload availability, data durability, and cost within the limits designed for the application.

9 Conclusion

This practical demonstrates a basic monitoring workflow for an EC2 resource. CloudWatch metrics provide a time series for instance CPU utilization, and a CloudWatch alarm can evaluate that metric against a threshold. SNS provides a separate mechanism for delivering alarm notifications to a confirmed email endpoint. An EC2 stop action can also be configured directly on the alarm, while a CloudWatch dashboard provides a visual summary of the metric.

The experiment also shows why configuration must be distinguished from execution. The documented alarm remained in `INSUFFICIENT_DATA`; this neither proves a threshold breach nor demonstrates that an EC2 action occurred. Reviewing metric availability, alarm state, missing-data behavior, and action settings is essential before relying on automated responses. Regular monitoring helps operators understand resource use, detect conditions that need attention, and manage cloud infrastructure more deliberately.

10 Glossary

Amazon CloudWatch AWS service for monitoring resource and application data, including metrics, alarms, and dashboards.

Amazon EC2 AWS service that provides resizable virtual compute instances.

Metric A named, time-ordered measurement of a resource or application.

Namespace A grouping that identifies the source or category of related metrics, such as `AWS/EC2`.

CPUUtilization EC2 metric expressing physical CPU use as a percentage.

Per-Instance Metrics EC2 metrics associated with an individual instance, commonly identified by its instance ID dimension.

Statistic A calculation that summarizes metric samples over a period, such as Average, Minimum, or Maximum.

Average The arithmetic mean of the metric samples in the selected period.

Period The interval over which metric samples are grouped to produce a datapoint.

Datapoint A metric value for a particular time or aggregation period.

Threshold A comparison boundary used by an alarm to classify metric values.

Static Threshold A fixed threshold value entered by the user.

CloudWatch Alarm A resource that evaluates a metric condition and maintains a state, with optional configured actions.

OK State Alarm state indicating that evaluated data is not breaching the configured condition.

ALARM State Alarm state indicating that the configured number of evaluated datapoints breach the condition.

Insufficient Data State used when CloudWatch lacks enough usable data to determine whether the alarm is OK or ALARM.

Amazon SNS AWS publish-and-subscribe messaging service that distributes messages to subscribed endpoints.

SNS Topic A named SNS message destination to which publishers send messages and endpoints subscribe.

Subscription A connection between an SNS topic and a receiving endpoint.

Email Endpoint An email address subscribed to receive messages from an SNS topic after confirmation.

Service-Linked Role An IAM role defined for an AWS service to perform supported tasks on a user's behalf.

CloudWatch Dashboard A customizable display that organizes monitoring information in widgets.

Dashboard Widget A panel on a dashboard, such as a metric graph, number, or text block.

Public Dashboard Sharing Sharing a dashboard so that anyone with its link can view it without AWS authentication.

11 References

The following official AWS documentation was consulted for service concepts and configuration behavior (accessed November 3, 2026). Console labels can change over time; use the live AWS console and current documentation when performing the practical.

References

[1] Amazon Web Services, *What is Amazon CloudWatch?*, https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html.

[2] Amazon Web Services, *View available metrics*, https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/viewing_metrics_with_cloudwatch.html.

[3] Amazon Web Services, *CloudWatch metrics that are available for your instances*, https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/viewing_metrics_with_cloudwatch.html.

[4] Amazon Web Services, *Using Amazon CloudWatch alarms*, https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch_Alarms.html.

[5] Amazon Web Services, *Configuring how CloudWatch alarms treat missing data*, https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/alarms-and-missing-data.html.

[6] Amazon Web Services, *Alarm actions*, https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/alarm-actions.html.

[7] Amazon Web Services, *Notifying users on alarm changes*, https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Notify_Users_Alarm_Changes.html.

[8] Amazon Web Services, *Using service-linked roles for CloudWatch*, https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/using-service-linked-roles.html.

[9] Amazon Web Services, *Create alarms that stop, terminate, reboot, or recover an instance*, https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/UsingAlarmActions.html.

[10] Amazon Web Services, *Amazon SNS email notifications*, https://docs.aws.amazon.com/sns/latest/dg/sns-email-notifications.html.

[11] Amazon Web Services, *Creating a customized CloudWatch dashboard*, https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/create_dashboard.html.

[12] Amazon Web Services, *Sharing a CloudWatch dashboard publicly*, https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/share-cloudwatch-dashboard-public.html.

[13] Amazon Web Services, *Sharing CloudWatch dashboards*, https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/cloudwatch-dashboard-sharing.html.